

Duvar

Point-Based Neural Mapping Framework

License: GNU General Public License v3.0
Architecture: PBNM-Flow (Point-Based Neural Mapping)
Reference model: TinyLlama-1.1B-Chat-v1.0
Runtime: Google Colab T4 GPU (16 GB VRAM)

1. Overview

Duvar (Turkish: Wall) is an experimental neural architecture that replaces standard sequential layer execution with a pointer-based directed graph. Instead of every layer blindly passing its output to the next in a fixed chain, each layer holds a `target_ptr` — an explicit pointer to its successor. This transforms the computation graph from a rigid pipeline into a reconfigurable graph of independent nodes called bricks.

The project began as an attempt to reduce model size via INT8 quantization. It achieves that goal (~2x compression on linear weights). But the more significant finding is structural: the pointer graph makes the execution path of a given input an explicit, inspectable artifact rather than an implicit consequence of tensor flow.

2. Architecture

2.1 Core Components

Component	Role
PointEntry	Fundamental data container: weight pointer + target layer ID
DuvarLinear	Drop-in nn.Linear replacement with INT8 quantization and routing pointer
DuvarGraph	Directed pointer graph mapping each layer to its successor
Triton Kernels	Fused FP16 operations: SiLU, GeLU, RMSNorm, INT8 dequantization

2.2 The Brick Abstraction

A `PointEntry` is the atom of the Duvar system — a brick: a self-contained unit holding a reference to its weight tensor and a pointer to the next brick (`target_layer`). A `DuvarLinear` wraps a standard linear operation and adds per-row INT8 quantization and a `next_layer_ptr` field identifying its successor. The full model becomes a `DuvarGraph`: a directed graph where traversal order is declared via pointers, not fixed Python module nesting.

2.3 Quantization Scheme (INT8, not FP8)

For each row r of a weight matrix:

```
scale[r] = (max(w[r]) - min(w[r])) / 255 index[r] = round((w[r] - min(w[r])) /  
scale[r]) in [0, 255] Reconstruction: w[r] = index[r] * scale[r] + min(w[r])
```

■ *Terminology: weights are stored as uint8 (8-bit unsigned integer, 256 quantization levels per row). This is INT8 quantization, not FP8. FP8 refers to 8-bit floating-point formats (e4m3/e5m2) used in hardware such as the H100. These are architecturally distinct. The matmul runs in FP16 after dequantization.*

3. Triton Kernels

Five fused kernels replace the standard PyTorch operations in the hot path. Each falls back to a pure-PyTorch implementation (identical mathematics) if Triton is unavailable.

Kernel	Operation	Benefit
duvar_dequant	Per-row INT8 $\rightarrow$ FP16	Fused to avoid a separate read-write pass over weights
duvar_silu	$x \cdot \text{sigmoid}(x)$	Avoids materialising full FP32 intermediate tensor
duvar_gelu	$0.5x(1 + \tanh(\dots))$	Numerically stable tanh via sigmoid identity
duvar_relu	$\max(x, 0)$	Baseline activation for ablation studies
duvar_rmsnorm	$x / \sqrt{\text{mean}(x^2) + \text{eps}} \cdot w$	Row-parallel normalisation

4. What the Pointer Graph Does (and Does Not Do)

This is the most important section to understand clearly before drawing conclusions from benchmark results.

What it does

- DuvarGraph constructs and stores a full directed pointer table over all linear layers.
- Every DuvarLinear instance carries a next_layer_ptr string identifying its declared successor.
- This table is saved to duvar_metadata.json and is the structural precondition for dynamic routing.
- The pointer table enables future sparse execution: only activate the bricks a given input needs.

What it does not yet do

- In the current notebook, the model still executes via HuggingFace's standard model.generate() forward pass.
- The pointer graph is metadata — it is not yet wired into the runtime dispatch loop.
- Computation order is still determined by PyTorch's module hierarchy, not pointer traversal.

This is not a flaw in the architecture; it is an honest statement of implementation status. The pointer table is the foundation layer. The runtime router — the part that enables sparse execution, live brick bypass, and topology-driven gains — is the next engineering step. It requires a custom generation loop or a full PBNM runtime, neither of which fits on a T4 Colab notebook.

5. Performance

5.1 Size

- Linear weight compression: ~2x (FP16 → INT8 per-row)
- Non-linear components (embeddings, norms, tokenizer) are not quantized and contribute overhead
- Total file size compression is slightly less than 2x due to these unquantized components

5.2 Speed — What the Numbers Actually Mean

The speed picture is nuanced and the notebook is honest about the trade-offs.

- Weights are stored in INT8 and dequantized to FP16 at every forward pass before the matmul. This is deliberate: hardware matmul units are faster in FP16 than INT8 on most GPU architectures.
- The net effect: smaller memory footprint reduces VRAM bandwidth pressure during weight loading, but dequantization adds a Triton kernel launch per layer.
- On memory-bandwidth-constrained hardware (T4), the bandwidth savings from loading INT8 instead of FP16 can outweigh the dequantization overhead — which is what the speed improvement reflects.
- Cell 6 (FP16 baseline) and Cell 9 (Duvar INT8) both run the same prompt at max_new_tokens=200 and measure wall-clock time. Cell 10 prints both side by side — this is a real comparison. Results (~12s baseline vs ~11s Duvar on T4) confirm that INT8 bandwidth savings outweigh the per-layer dequantization cost.
- Cell 12 additionally benchmarks Duvar per-token latency using torch.cuda.Event across 10 iterations for sub-millisecond precision. Running the same benchmark against the FP16 baseline would make the numbers fully rigorous.

■ *Cell 10 uses do_sample=True (stochastic outputs). Fixing the seed and running Cell 12's CUDA-event benchmark for both models would make the latency claim watertight. Jaccard similarity is a word-overlap proxy, not a formal quality metric.*

6. Explainability (XAI) Properties

Duvar's architecture is structurally more amenable to explainability than standard transformer black-boxes. This is a design consequence, not a post-hoc addition.

Property	How Duvar Enables It
Route Traceability	Each brick holds an explicit pointer to its successor. The path a given input takes through the model can be recorded as a concrete sequence of layer identifiers.

Property	How Duvar Enables It
Modular Ablation	Disabling a layer requires only redirecting its target_ptr — no retraining, no masking, no weight surgery. Real-time ablation studies become trivially cheap.
Spatial Activation Mapping	The pointer table forms an opportunity map. Bricks visited frequently by a class of inputs are structurally distinguishable from bypassed bricks.
Granular Interpretability	Each brick can be labelled with a semantic role. If a reasoning-specialised brick is not visited, the model is provably operating in a surface-pattern mode.

Existing XAI tools (LIME, SHAP) estimate attribution from outside the model. Duvar exposes the execution path from inside.

7. Comparison with Prior Work

Prior Art	Similarity	Key Difference
Switch Transformers / MoE	Dynamic dispatch over sub-networks	MoE works on a fixed backbone; Duvar breaks the backbone into a reconfigurable directed graph
LangGraph	Nodes and edges for flow control	LangGraph routes at the agent/application level; Duvar routes at the tensor/layer level
Deep Equilibrium Models	Iterative / non-sequential computation	DEQ uses fixed-point iteration; Duvar's bricks are independent and hot-swappable

8. Theoretical Claims

- Topology over arithmetic: replacing brute-force matmul traversal with lookup-driven routing reduces active operation count — but only once the runtime router is implemented and sparse paths are activated.
- Bandwidth over latency: INT8 storage reduces the volume of data moved from VRAM to SM, which matters more than raw compute speed on bandwidth-constrained hardware.
- Sparse execution potential: only the bricks relevant to a given input need to be activated. This is the structural precondition — the runtime to exploit it is future work.

9. Honest Limitations

- Runs on T4. Theoretical gains from topology-driven routing become measurable at scale (70B+ parameter models, distributed inference, hardware-aware routing).
- The pointer graph is currently metadata. Runtime routing is not yet implemented.
- Cell 10 comparison uses do_sample=True. Fixing the seed and running Cell 12's CUDA-event benchmark against both FP16 and Duvar would make the latency numbers fully rigorous.

- INT8 quantization introduces small reconstruction errors (max dequant error < 0.001 in FP16 precision bounds — verified in Cell 11).
- Jaccard similarity is a word-overlap proxy. A proper quality evaluation requires perplexity on a held-out corpus or a downstream task benchmark.

10. Usage

Requirements

```
transformers==4.40.0 accelerate sentencepiece protobuf huggingface_hub triton #  
optional but recommended
```

Cell Execution Order (Google Colab T4)

Cell	Action	Notes
1	Environment setup	~60s, run once
2	Imports and GPU verification	Checks Triton availability
3	Download TinyLlama-1.1B	~2.2 GB; cached after first run
4	Compile Triton kernels	Falls back to PyTorch if Triton unavailable
5	Load Duvar architecture classes	DuvarLinear, PointEntry, DuvarGraph
6	FP16 baseline inference	Sets t_baseline and baseline_text
7	Convert model to Duvar	INT8 quantization + pointer graph construction
8	Save converted model	Saves to /content/duvar_output/
9	Duvar inference	Uses same prompt as Cell 6
10	System report	Prints size, quality, and speed summary
11	Kernel accuracy verification	Max error assertions for all Triton kernels
12	Latency benchmark	10-iteration per-token ms benchmark (Duvar only)

Output Files

File	Contents
duvar_weights.pt	Full model state dict with INT8 linear weights
duvar_metadata.json	Pointer graph table, compression stats, architecture info
tokenizer_config.json + supporting files	Saved tokenizer